

Python — La visualisation de données

Document 00 — Introduction, architecture et vision globale

Série « Visualisation » · document d'ouverture

Table des matières

- 1. Le pipeline de visualisation
- 2. Le paysage des bibliothèques
 - 2.1 Les bibliothèques au programme de la série
 - 2.2 Tableau comparatif
- 3. Les types de visualisation
- 4. Choisir la bonne bibliothèque selon le besoin
- 5. L'architecture d'un projet de visualisation
 - 5.1 La séparation des couches
 - 5.2 Un exemple de structure de projet
- 6. Les sous-modules : importer ce dont on a besoin
- 7. Plan de la série
- Récapitulatif
- Questions de vérification

1. Le pipeline de visualisation

Visualiser n'est jamais une étape isolée : c'est l'aboutissement d'une chaîne. Comprendre cette chaîne aide à savoir où placer chaque outil. On peut la voir en quatre étapes.

DONNEES (sources)	->	TRAITEMENT (preparation)	->	VISUALISATION (trace)	->	SORTIE (diffusion)
CSV, API, base SQL, flux temps reel		Pandas/NumPy : nettoyer, agregger, calculer		Matplotlib, Seaborn, Plotly, PyQtGraph...		Image (PNG/PDF), page web (HTML), dashboard, fichier

- 1. Données.** Tout part de sources brutes : un CSV de prix, une API de marché, une base SQL, un flux temps réel. (C'est l'objet du document 6 de la série Python.)
- 2. Traitement.** On nettoie, on agrège, on calcule des indicateurs avec Pandas et NumPy (document 5). Une bonne visualisation repose sur des données déjà propres : on ne trace pas des données sales.
- 3. Visualisation.** On transforme les données préparées en représentation visuelle. C'est le cœur de cette série, et c'est là qu'interviennent les différentes bibliothèques.
- 4. Sortie.** Le résultat est diffusé : une image figée pour un rapport, une page web interactive, un tableau de bord en direct. Le format de sortie dépend de l'usage, et il oriente le choix de la bibliothèque.

Pourquoi ça compte. Idée directrice : la bibliothèque de visualisation se choisit surtout en fonction de l'ETAPE 4 (la sortie voulue). Une image pour un PDF n'appelle pas le même outil qu'un dashboard interactif ou un flux temps réel. C'est le fil conducteur de tout ce document.

2. Le paysage des bibliothèques

2.1 Les bibliothèques au programme de la série

L'écosystème Python compte des dizaines de bibliothèques de visualisation. La série se concentre sur celles qui sont réellement utiles, en particulier dans un contexte de données financières et de trading.

Les fondations généralistes :

- **Matplotlib** — la fondation de tout l'écosystème. Contrôle total, tous types de graphiques, sortie statique. Les autres bibliothèques s'appuient dessus.
- **Pandas (.plot)** — tracer directement depuis un DataFrame, en une ligne. Idéal pour explorer vite.
- **Seaborn** — bâtie sur Matplotlib ; graphiques statistiques élégants (distributions, corrélations) avec un style soigné.
- **Plotly** — graphiques interactifs (zoom, survol, info-bulles) pour le web et les tableaux de bord ; sortie HTML.

Les spécialisées (utiles en trading) :

- **PyQtGraph** — tracé haute performance et basse latence, conçu pour le temps réel : flux de marché en direct, capteurs, applications de bureau. Là où Matplotlib peine (rafraîchissement rapide), PyQtGraph excelle.
- **Matplotlib 3D (mplot3d)** — visualisation en trois dimensions : surfaces, nuages 3D, courbes paramétriques. Utile pour les surfaces de volatilité, les relations à trois variables.

2.2 Tableau comparatif

Une vue d'ensemble pour situer chaque bibliothèque selon ce qui compte vraiment : le type de sortie, l'interactivité, la performance, et le cas d'usage type.

Bibliothèque	Sortie	Forces et usage type
Matplotlib	Statique (PNG, PDF, SVG)	Contrôle total, tous types de graphiques. La fondation. Idéale pour les rapports figés.
Pandas .plot	Statique (via Matplotlib)	Exploration ultra-rapide depuis un DataFrame. Une ligne de code.
Seaborn	Statique (via Matplotlib)	Statistiques et style élégant : distributions, corrélations, catégories.
Plotly	Interactif (HTML)	Zoom, survol, dashboards web. Inspection dynamique des données.
PyQtGraph	Interactif (fenêtre Qt)	Temps réel, haute fréquence de rafraîchissement, basse latence. Flux de marché.
Matplotlib 3D	Statique (3D projeté)	Surfaces, nuages et courbes en trois dimensions.

3. Les types de visualisation

Au-delà des bibliothèques, il est utile de classer les visualisations par **nature**. Chaque type appelle des outils différents et répond à un besoin distinct.

Type	Outil typique	Quand l'utiliser
Statique	<i>Matplotlib, Seaborn</i>	Rapport, PDF, document imprimé : une image figée qui ne change pas.
Interactif	<i>Plotly</i>	Exploration, dashboard web : l'utilisateur zoome, survole, filtre.
Temps réel	<i>PyQtGraph, Matplotlib animation</i>	Flux de données en direct : prix, capteurs, monitoring.
3D	<i>Matplotlib mplot3d, Plotly 3D</i>	Trois variables, surfaces, géométrie : surface de volatilité par ex.
Big data	<i>Datashader (hors série)</i>	Des millions de points : agrégation par pixel plutôt que tracé point par point.
Géospatial	<i>Folium, GeoPandas (hors série)</i>	Données sur une carte : positions, régions.

La série couvre les quatre premiers types (statique, interactif, temps réel, 3D), les plus pertinents en finance. Les deux derniers (big data massif, géospatial) sont mentionnés pour situer le paysage, mais sortent du périmètre.

4. Choisir la bonne bibliothèque selon le besoin

La question centrale n'est pas « quelle est la meilleure bibliothèque » (aucune ne l'est dans l'absolu), mais « laquelle pour CE besoin ». Voici une démarche de décision.

Quelle est la SORTIE voulue ?

Une image pour un rapport/PDF ?

-> Matplotlib (ou Seaborn pour le style, Pandas pour aller vite)

Un graphique web ou un tableau de bord ou l'on explore ?

-> Plotly

Un affichage qui se met à jour en direct (flux de marche) ?

-> PyQtGraph (haute performance) ou Matplotlib animation

Une représentation en trois dimensions ?

-> Matplotlib 3D (statique) ou Plotly (3D interactif)

Juste un coup d'oeil rapide pendant l'analyse ?

-> Pandas .plot (une ligne, directement sur le DataFrame)

En pratique, on combine : on **explore** avec Pandas/Seaborn, on **peaufine** le rendu final avec Matplotlib, on passe à **Plotly** quand l'interactivité aide, et à **PyQtGraph** quand la vitesse de rafraîchissement devient critique. Ces outils sont complémentaires, pas concurrents.

Pourquoi ça compte. Piège classique : vouloir tout faire avec une seule bibliothèque. Un bon réflexe est de séparer l'exploration (rapide, jetable) du livrable (soigné, durable), et de choisir l'outil adapté à chacun.

5. L'architecture d'un projet de visualisation

5.1 La séparation des couches

Dans un vrai projet, on ne mélange pas le code qui charge les données, celui qui les trace, et celui qui exporte. On sépare en **couches**, ce qui rend le code lisible, testable et réutilisable.

Couche	Rôle	Exemple d'outils
Données	Charger et fournir les données brutes	read_csv, API, SQL (doc 6)
Traitement	Nettoyer, agréger, calculer	Pandas, NumPy (doc 5)
Visuel	Construire les graphiques	Matplotlib, Seaborn, Plotly
Export	Produire la sortie finale	savefig, write_html, to_pdf
Interface	Présenter à l'utilisateur (optionnel)	Dashboard, fenêtre, rapport

L'intérêt : si la source de données change (un nouveau courtier), seule la couche données bouge ; les graphiques restent identiques. Si on veut exporter en PDF au lieu de PNG, seule la couche export change. Chaque couche a une seule responsabilité.

5.2 Un exemple de structure de projet

Une organisation de fichiers typique pour un petit projet de visualisation, reflétant ces couches.

```
projet_visualisation/  
  donnees.py      # couche donnees : charger les prix (CSV, API)  
  traitement.py   # couche traitement : nettoyer, calculer les indicateurs  
  graphiques.py   # couche visuel : fonctions qui construisent les figures  
  export.py       # couche export : sauvegarder en PNG/PDF/HTML  
  main.py         # orchestre : donnees -> traitement -> graphiques -> export  
  
# Exemple d'orchestration dans main.py  
from donnees import charger_prix  
from traitement import calculer_indicateurs  
from graphiques import tracer_prix  
from export import sauvegarder
```

```
df = charger_prix("ES")          # 1. donnees  
df = calculer_indicateurs(df)     # 2. traitement  
fig = tracer_prix(df)             # 3. visuel  
sauvegarder(fig, "rapport_es.pdf") # 4. export
```

Chaque fichier correspond à une couche. `main.py` se contente d'orchestrer la chaîne, sans contenir de logique. On retrouve le pipeline de la section 1, traduit en code. Cette discipline, héritée des bonnes pratiques du document 7 (qualité), distingue un projet maintenable d'un script jetable.

6. Les sous-modules : importer ce dont on a besoin

Chaque bibliothèque est en réalité un ensemble de **sous-modules** qu'on importe séparément selon le besoin. On n'importe pas « tout Matplotlib » : on importe `pyplot` pour tracer, `dates` pour formater un axe temporel, etc. Connaître ces sous-modules évite de réinventer ce qui existe déjà.

Matplotlib — les sous-modules courants :

Sous-module	Alias usuel	À quoi il sert
<code>matplotlib.pyplot</code>	<code>plt</code>	Interface principale : créer figures et axes, tracer.

Sous-module	Alias usuel	À quoi il sert
<code>matplotlib.gridspec</code>	<i>gridspec</i>	Dispositions de subplots complexes (grilles inégales, fusions).
<code>matplotlib.dates</code>	<i>mdates</i>	Formater et graduer un axe de dates (séries temporelles).
<code>matplotlib.ticker</code>	<i>mticker</i>	Contrôle fin des graduations (% , milliers, log).
<code>matplotlib.colors</code>	<i>mcolors</i>	Couleurs et normalisation (échelles de couleur).
<code>matplotlib.cm</code>	<i>cm</i>	Colormaps (palettes de couleurs continues).
<code>matplotlib.patches</code>	<i>mpatches</i>	Formes géométriques (rectangles, flèches, zones).
<code>matplotlib.image</code>	<i>mpimg</i>	Lire et afficher des images existantes (PNG...).
<code>matplotlib.animation</code>	<i>animation</i>	Animations et export GIF/MP4 (temps réel).
<code>mpl_toolkits.mplot3d</code>	<i>Axes3D</i>	Tracés en trois dimensions (projection '3d').

```
# Exemples d'imports typiques de Matplotlib
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.dates as mdates
import matplotlib.ticker as mticker
import matplotlib.colors as mcolors
import matplotlib.image as mpimg
from matplotlib.animation import FuncAnimation
from mpl_toolkits.mplot3d import Axes3D
```

Les autres bibliothèques — imports typiques :

Bibliothèque	Import typique	Usage
Pandas	<i>import pandas as pd</i>	Tracé intégré via <code>df.plot(...)</code> .
Pandas	<i>from pandas.plotting import scatter_matrix</i>	Outils spéciaux (matrice de nuages, autocorrélation).
Seaborn	<i>import seaborn as sns</i>	API statistique classique.
Seaborn	<i>import seaborn.objects as so</i>	Nouvelle API par objets (composition).
Plotly	<i>import plotly.express as px</i>	API rapide (une fonction par type).
Plotly	<i>import plotly.graph_objects as go</i>	API détaillée (contrôle fin).
Plotly	<i>from plotly.subplots import make_subplots</i>	Plusieurs graphiques dans une figure.
PyQtGraph	<i>import pyqtgraph as pg</i>	Tracé temps réel haute performance.
Pillow	<i>from PIL import Image, ImageDraw, ImageFont</i>	Ouvrir, dessiner et écrire sur des images.

Pourquoi ça compte. Bon réflexe : importer le sous-module précis dont on a besoin, avec son alias conventionnel (*plt*, *mdates*, *px*...). Cela rend le code lisible pour quiconque connaît l'écosystème, et c'est la convention qu'on suit dans toute la série.

7. Plan de la série

Voici les documents prévus. Chacun (sauf celui-ci et la référence finale) traite une bibliothèque en reprenant le **même squelette d'exemples** : image simple, graphique complet (titres, axes, légende),

types de tracés, styles, plusieurs graphiques, texte combiné aux graphiques, et sauvegarde. Cette structure parallèle permet de comparer les bibliothèques à contenu équivalent.

Document	Contenu
00 (ce document)	Introduction, architecture, pipeline, choix d'outil.
10 – Matplotlib	La fondation : Figure/Axes, subplots, GridSpec, styles, annotations, texte+graphique, export.
20 – Pandas	Tracer depuis un DataFrame ; exploration rapide.
30 – Seaborn	Graphiques statistiques et style élégant.
40 – Plotly	Graphiques interactifs et tableaux de bord.
50 – PyQtGraph	Visualisation temps réel haute performance.
60 – Matplotlib 3D	Surfaces, nuages et courbes en trois dimensions.
70 – Images et texte	Afficher des images, combiner texte et graphiques, overlays.
999 – Cheat sheets	Référence : colormaps, marqueurs, styles, rcParams, patterns fréquents.

L'ordre est pensé pour progresser : on maîtrise d'abord Matplotlib (la fondation), puis les couches qui s'appuient dessus (Pandas, Seaborn), puis l'interactif (Plotly), le temps réel (PyQtGraph) et la 3D, avant les sujets transversaux (images, référence).

Récapitulatif

- 1. Le pipeline** : données → traitement → visualisation → sortie. La visualisation est l'aboutissement d'une chaîne ; le format de sortie oriente le choix de l'outil.
- 2. Les bibliothèques** : Matplotlib (fondation, statique), Pandas (.plot, rapide), Seaborn (statistique, élégant), Plotly (interactif), PyQtGraph (temps réel), Matplotlib 3D (trois dimensions).
- 3. Les types** : statique, interactif, temps réel, 3D — chacun avec son outil. On choisit selon la sortie voulue.
- 4. L'architecture** : séparer les couches (données, traitement, visuel, export, interface) pour un projet lisible et maintenable.
- 5. Les sous-modules** : on importe le sous-module précis dont on a besoin (pyplot, gridspec, dates, mplot3d...) avec son alias conventionnel.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Q1. Quelles sont les quatre étapes du pipeline de visualisation ?

Réponse. Données (sources brutes) → Traitement (nettoyer, agréger, calculer) → Visualisation (tracer) → Sortie (diffuser : image, web, dashboard). La visualisation est l'aboutissement d'une chaîne.

Q2. Qu'est-ce qui oriente principalement le choix d'une bibliothèque ?

Réponse. La sortie voulue (l'étape 4). Une image pour un PDF appelle Matplotlib ; un dashboard interactif appelle Plotly ; un flux temps réel appelle PyQtGraph.

Q3. Quelle bibliothèque pour un flux de marché en direct, et pourquoi ?

Réponse. PyQtGraph : il est conçu pour la haute performance et la basse latence, là où Matplotlib peine à rafraîchir rapidement. (Matplotlib animation reste une option plus simple mais moins performante.)

Q4. Pourquoi séparer un projet de visualisation en couches ?

Réponse. Pour que chaque couche ait une seule responsabilité : si la source de données change, seule la couche données bouge ; si le format d'export change, seule la couche export bouge. Le code reste lisible et maintenable.

Q5. Quelle est la différence entre matplotlib.pyplot et matplotlib.dates ?

Réponse. pyplot (plt) est l'interface principale de tracé (créer figures et axes, tracer). matplotlib.dates (mdates) sert spécifiquement à formater et graduer un axe de dates, essentiel pour les séries temporelles.

Q6. Quand utiliser Pandas .plot plutôt que Matplotlib directement ?

Réponse. Pour un coup d'œil rapide pendant l'analyse : .plot() trace en une ligne depuis un DataFrame. Pour un livrable soigné, on revient à Matplotlib (que .plot utilise de toute façon en coulisse).

Document suivant (10) : *Matplotlib, la fondation — Figure et Axes, subplots et GridSpec, styles, annotations, texte combiné aux graphiques, et export.*